

Digital Twin Data Broker with Assisted Mapping into a Knowledge Base

Thomas Schmeyer¹, Kai Krämer¹, Anna-Lena Peh², Boris Brandherm¹,
Margarita Chikobava¹, and Gian-Lucca Kiefer¹

¹ German Research Center for Artificial Intelligence, Cognitive Assistance Systems,
66123 Saarbrücken, Germany,

`{name.lastname}@dfki.de`

² Fraunhofer IOSB, Angewandte Systemtechnik, 98693 Ilmenau, Germany
`{name.lastname}@iosb-ast.fraunhofer.de`

Abstract. The frequent usage of digital twins to communicate between physical objects is resulting in more complex cyber-physical systems. To simplify the individual components' integration and to optimize their usage, a data broker is being developed. Therefore, digital twins need to be semantically organized in an ontology that provides the advantage of reasoning methods. An assisted workflow is being developed to automatically enter subgraphs into an ontology. As a digital twin representation, the Asset Administration Shell format is used to have an international standard technology. Based on this, a new domain-specific language is developed, allowing experts to configure the generation process. This process maps the digital twin's information into a graph representation of the ontology. The preconfigured generation process enables the user to efficiently register new digital twins without having expert knowledge of the underlying ontology. Additionally, a Large Language Model vector embedding and text reasoning support is implemented analysing the digital twin to create entity suggestions. The presented data broker is an automation tool for bridging the gap between semantic descriptions and digital twin formats in order to unite the advantages of both representations.

Keywords: Digital Twin · Asset Administration Shell · Ontology · Automation · Cyber-Physical System

1 Introduction

This thesis aims to make digital twins semantically interoperable by using the Asset Administration Shell (AAS) [1] format with a domain-specific language to enable a semi-automatic mapping to the Open Energy Ontology (OEO) [2]. This mapping ensures a precise and uniform description of resources and thus enables the efficient use and searchability of the digital twins on a digital marketplace for energy-related goods.

Industry 4.0 needs software solutions to create more flexible and effective workflows. However, the heterogeneous integration of these software solutions is

leading to increasingly complex cyber-physical systems. Managing these systems requires in-depth specialist knowledge. This also affects the energy sector.

An intelligent and interactive energy system is needed to enable the efficient use of energy from volatile renewable energy sources such as solar panels and wind turbines and through electric heating systems or electric personal transport. This includes the generation of synthetic data of households [3] and forecasting methods [4] to estimate the energy consumption and production of electrical grids. An approach combining these methods into one cyber-physical system is used to protect only sparsely monitored low-voltage grids [5].

The European Commission released a strategy to accelerate the digital transformation of the European Energy System [6]. Especially the innovation of Digital Twin (DT) technology and Artificial Intelligence of Things (AI-IoT) of the Horizon Europe program should be transferred to the energy infrastructure. There is a whole range of DT applications with various use cases in different energy sector industries [7] that need to be managed in a system critical infrastructure and in heterogeneous systems.

Hence, there is a need to semantically annotate digital twins for their specific task. This can simplify the integration of digital twins into production. It also helps to find suitable digital twins on an external marketplace for digital goods. Therefore a data broker application is developed that provides an assistance system to transfer a digital representation of an asset automatically into an ontology, see Figure 1. The used base ontology in this use case scenario is the OEO [2]. Some new properties extend the OEO to integrate descriptions of DTs. Enhancing the scalability of the data broker, the extended ontology is transferred from the OWL format to a graph database, see section 3.

The DTs must be integrated interoperably into existing software systems. An international standard technology for describing and exchanging information about DTs is the AAS [1]. An AAS-JSON file is used as base representation of DTs managed by the broker. Such a shell contains the information and functional aspect of an asset in submodels. An assisted mapping of submodels to the ontology is developed.

Therefore, suitable asset descriptions are extracted. They are analyzed via a large language model (LLM) and classical text mining algorithms to propose to the user some suggestions for ontology classes. This is described in section 4.1. Then the submodels of the AAS file are matched against saved template models, with a configured automated graph generation. This generation is configured by a newly developed domain-specific language (DSL). If such a match is recognized, the configured DSL code is executed to generate a subgraph. This is added to the knowledge base. The process takes already existing entries into account to prevent data duplication.

In the next section, a summary is given on related work. It is followed by a detailed description of the OEO with the new developed extension in sections 2 and 3 to explain the application's theoretical background. In section 4 the data broker application is described in detail, focusing on the assistance system. Followed by a discussion in section 5 with an outlook.

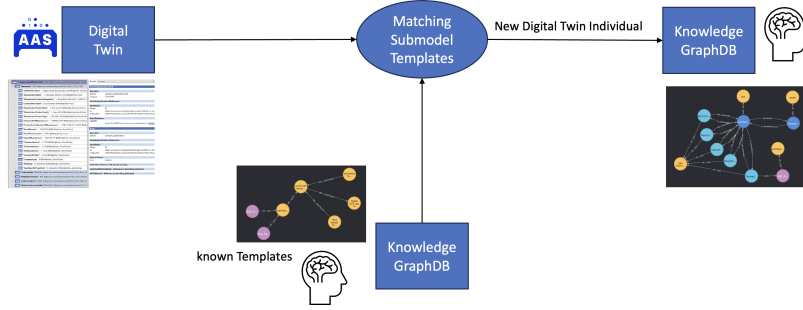


Fig. 1. The main idea of the assistance system.

2 Related Work

In [8] a so-called Semantic Asset Administration Shell (SAAS) is presented that maps the AAS model to Resource Description Framework (RDF). This is used to build a validation model of AAS exchange data specified in the Shapes Constraint Language (SHACL).

An approach to enable semantic interoperability with an ontology is demonstrated in [9]. The Authors use a transformation from OWL to Unified Modeling Language (UML) using a concept that maps between different transformations to generate AAS stereotypical descriptions. The AAS files are then annotated to this UML description and then vice versa remapped to an ontology.

Another approach, in which ontology is used as a semantic description, is shown in [10]. The authors transform RDF ontologies to AAS template files without losing the ontology URL reference. They provide an algorithm to rebuild the RDF graph from an instance file of an AAS and an algorithm to select such an annotated file with the standard AAS environment methods.

An approach to bridge the gap between different versions of control software is developed in [11]. The authors used a hierarchical domain-specific language that has direct access to the AAS environment to inject the right software version to control industrial machines.

In [12], the authors use an LLM to enrich some properties of a so-called semantic node with textual information. Therefore, they read a textual input of a user and extract the preliminary properties, like name, value, initial descriptions. This information is combined with an enhanced ECLASS search from a generative LLM Agent to a semantic node. As shown in [10], such a node can then be referenced by an AAS Submodel or used by other agents.

An approach that uses an AAS environment in combination with a graph database is presented in [13]. The authors used a specialized AAS hazard model and transferred them into a knowledge graph. Using the query language of the database they detect hazards in a worker environment.

3 OEO and AAS

The Open Energy Ontology (OEO) has been developed to facilitate collaboration and information exchange in the field of energy systems analysis. It was created with the aim of improving scientific exchange and reflecting the complexity of the research field. Ambiguous terminologies of energy systems analyses and related fields have been collected, linked and structured. Like ontologies in other fields, the OEO helps to replace heterogeneous data, different definitions and incompatible models. By annotating information with the OEO, it becomes reusable, searchable, exchangeable, comparable, linkable and semantically interoperable [2]. As far as possible, the OEO follows a taxonomic structure in which each class has exactly one parent class in a so-called monohierarchy.

The OEO is defined in expressive OWL2 semantics, which allows implicit knowledge to be inferred. The top-level ontology of the OEO is the Basic Formal Ontology (BFO) [15]. The OEO resources are being developed and are available on GitHub.

3.1 Extension of OEO by AAS Digital Twin Information

The natural OEO has no structure to describe the AAS technology. Therefore, an additional part is added, using a similar structure like [8] and [14] to describe AAS structure, see Figure 2. As base description the concepts of asset, AAS and submodels are added. An asset represents exactly one AAS. Such an AAS can have multiple submodels. It is defined that a submodel has only one AAS as a parent, see Figure 2. More detailed information can be saved as data properties in the database for further use. The new classes are set as subclasses of the 'content entity'-class and the connection is from the relative loose relation 'has information content entity' from the OEO. Submodels of a shell are also stored if they fulfill standards. Such standardized submodel templates are provided by the Industrial Digital Twin Association (IDTA), for example. The IDTA is a central point for standardization for digital twins to enhance interoperability and also provide such submodel templates, [16]. Such templates specify the structure of how certain submodels should look.

For scalability, the ontology was imported to a Neo4J graph database. Therefore, the Neosemantic plugin is used [17]. It allows the import of RDF and associated vocabularies like OWL to Neo4J Graph Databases. In order to simplify the import and have the possibility to change the underlying ontology, the import functionality is automatized via a shell script. So the application can adapted easily to another use case.

Another reason is the flexibility of stored objects. It enables a wider range of graph-based analyses, such as vector similarities, directly via the database. An example application is shown in section 4.1.

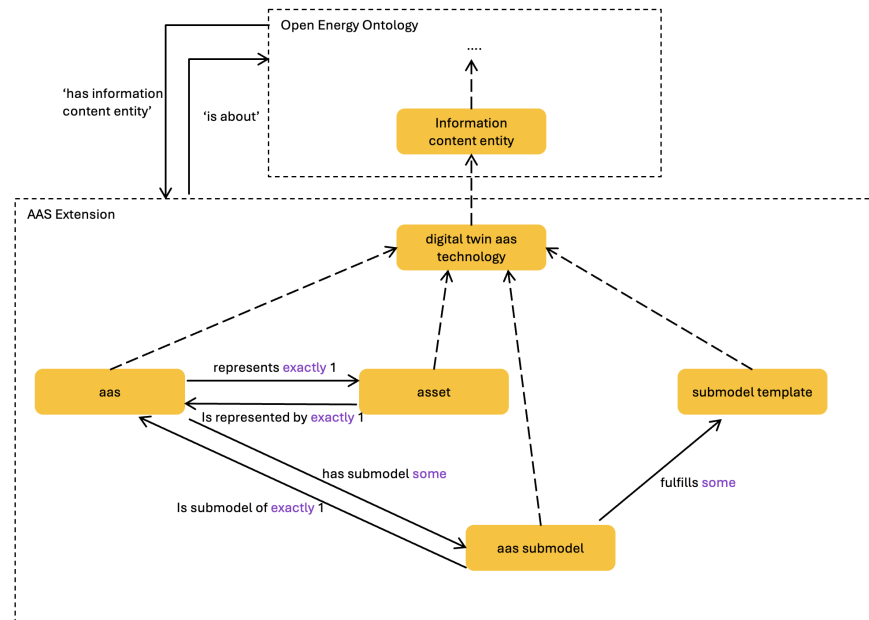


Fig. 2. Overview of the new developed extensions added to the OEO. The dotted arrows are 'is subclass of' properties. The slide arrows show the class description.

4 Digital Twin Data Broker

The application architecture is separated into backend, middleware and frontend, see Figure 3. The architecture is containerized, so that all parts are easily portable.

The backend contains the graph database storing the knowledge base of the OEO with the newly developed extensions. It also contains a large language model (LLM) to transform AAS asset descriptions and labels into a vector embedding, which allows fast similarity matching.

The middleware is designed as a REST service and has three main functionalities. A query service to provide and aggregate data about the stored DTs in the database for the user. Then there is an AAS register template service, which gives a domain ontology expert the possibility to configure the automatized generation of knowledge base subgraphs for predefined templates. The configuration is done by a newly developed DSL presented in section 4.3.

These configured templates are used in a DT register service to provide the user an assistance system to register DTs. It checks if the uploaded AAS file's submodels correspond to template standards. In addition, it also provides an interface to analyze AAS files to suggest to the user where to assign the asset to the knowledge graph. Additionally, an automatization was developed to transfer the properties of matched submodels into the knowledge base. The following section will focus on this automatization procedure/service.

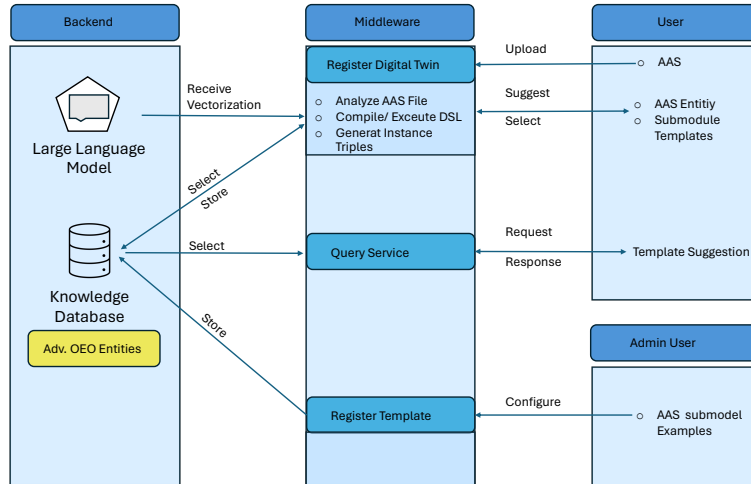


Fig. 3. Service Architecture Overview.

4.1 Labelmapping with Large Language Models and Jaccard Index

The functionality presented is intended to help an external user to correctly assign their digital twin in the ontology. A LLM is used to examine the uploaded AAS file of the digital twin. For this purpose, an interface was provided that segments the stored class label string into tokens and calculates the semantic embedding of the LLM as a vector. This is stored as a property of the class in the database so that it is available for further mathematical methods.

In order to suggest to the user the best possible matches, the descriptions of the asset are extracted from the uploaded file. Therefore, the standard description field of the AAS format is used, which is always available. Other descriptions of standardized formats are taken into account if available, such as the nameplates of the IDTA and HSU.

The description's embedding vectors are then computed with the LLM and matched against the label vectors with the cosine similarity

$$S_c(v_1, v_2) = \frac{\langle v_1, v_2 \rangle}{\|v_1\|_2 \cdot \|v_2\|_2}, \quad (1)$$

where $v_1, v_2 \in \mathbf{R}^n$ are embedding vectors with $n \in \mathbf{N}$, $\langle \cdot, \cdot \rangle$ the standard dot product and $\|\cdot\|_2$ the l_2 -norm to normalize the vectors [18]. The results of different descriptions of one OWL class label are then merged as mean value. The suggestions are then presented in descending order of the computed similarity value.

As an example of a description, a solar heat plant is analysed. It is described as "A solar heat plant is a heat plant that has solar heat units as parts." in a comment of the OEO. As LLM the pretrained BERT-base-uncased [19] without adaption of the weights is used to analyse the description. The result is presented in Table 1. Because of the lack of a large corpus of AAS sample descriptions an interface is implemented to get additional descriptions of the ontology itself. These embedding vectors are then computed and compared with the cosine similarity (1) to provide a guess of the accuracy of the used LLM. In order to test the abstraction ability of the LLM the descriptions are additionally modified by substituting the single words of a label by a placeholder like thing a , thing b This is done for all combinations.

Using the not adapted BERT-base-uncased model it shows that only 639 of 1145 entities has a hit in the first 20 suggested results. The best similarity match was about $0.9 \in [0; 1]$ with the natural description and about 0.6 in mean with replaced label word combinations.

The results show that there is a need to retrain a LLM to a specific scenario to enhance the accuracy of matching labeled entities. Because of that, the application provides the user, besides the manual search, some classical approaches. A Jaccard Index based algorithm has been implemented counting the matched words of the label. The result of the previous example is shown in Table 2.

When looking at the result tables 1 and 2 of the example above, both algorithms select the correct label. While the Jaccard algorithm naturally has advantages if the label is named directly in the description, COS-Similarity can

also make suggestions abstracted from the description using LLM word embedding.

Label	COS-Similarity
solar heat plant	0.609
solar heat unit	0.604
solar power plant	0.601
solar thermal heat technology	0.587
solar thermal power unit	0.578
solar thermal energy transformation	0.572
geothermal heat plant	0.568
solar energy	0.567
rooftop photovoltaic power plant	0.563
solar thermal power technology	0.560

Table 1. Resulting LLM suggestions of the label solar heat plant ordered by cos-similarity.

Label	Count
solar heat plant	7
heat plant	5
geothermal heat plant	5
solar heat unit	5
combined heat and power plant	5
solar thermal heat technology	5
solar power plant	4
plate heat exchanger	3
heat transfer unit	3

Table 2. Resulting jaccard suggestions for the user ordered by the count of words.

4.2 Class Description Mapping of OEO

In ontologies, classes can include general class axioms or descriptions, which are logical statement and readable in Protégé. In Neo4j databases, these statements become very difficult to read, see Figure 4 and 5. The Neo4j plugin Neosemantics provides some reasoning methods but no functionality to make these descriptions readable. In order to provide such class descriptions to users, a mapping algorithm is implemented solving the OWL class descriptions and equivalences properties. This uses the standardised web ontology referenced in [20], especially the class properties. As a simple example with two restrictions, the graph of the previous example of a solar heat pump is given in Figure 4. These restrictions are mapped to the following semantic sentences:

1. solar heat plant has part some solar heat unit
2. solar heat plant participates some solar thermal energy

A more complicated example is the graph of an electric vehicle, see Figure 5. Since it is a subgraph that describes the equivalences, the mapping algorithm must handle intersections to solve the logical expression. The equivalences and the restrictions are mapped to following semantic sentences:

1. electric vehicle is equivalent to (vehicle and has part some electric traction motor)
2. electric vehicle has energy input some electrical energy

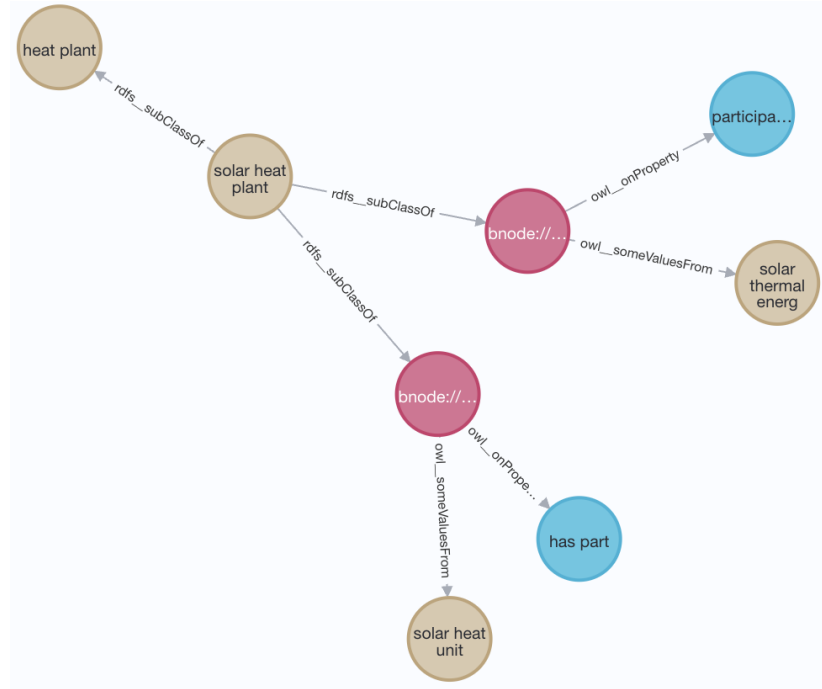


Fig. 4. Solar Heat Pump class descriptions as graph stored in the database.

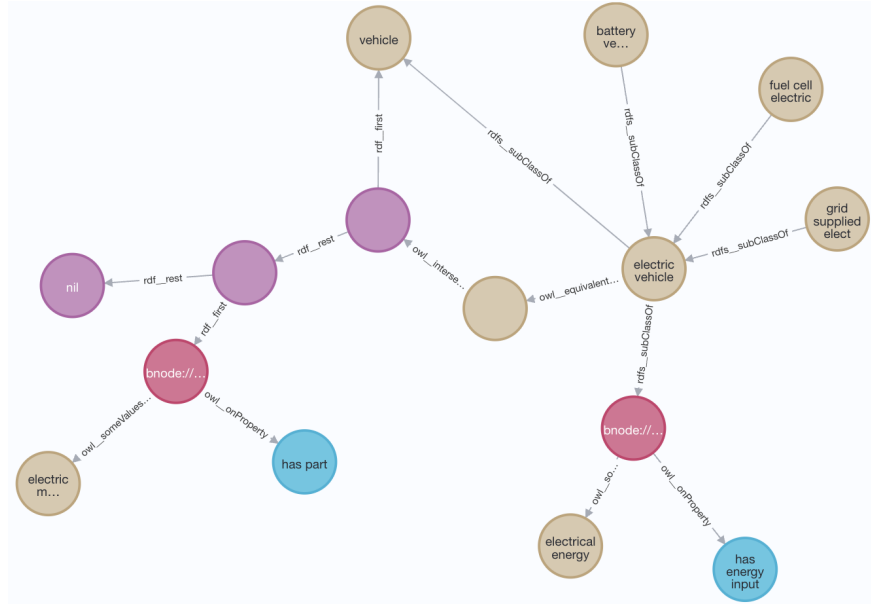


Fig. 5. Electric Vehicle class descriptions as graph stored in the database.

4.3 Domain Specific Language

In contrast to relational databases, which can be easily queried with SQL, there is currently no standardized query mechanism in the AAS. To address this limitation and simplify the configuration logic on a matching AAS templates, a DSL is developed. It analyse AAS files based on the hierarchical structure of the submodel templates. The DSL syntax is based on lambda expressions introduced with " $\Rightarrow$ ". The access operator is \$ followed by the variable name whose value is accessed. It serves as a placeholder for the object that is successively changed by the lambda operators. Without any variable name, the root object is accessed, in this use case an AAS submodel format.

One base functionality of the DSL is the usage of JMESPath expressions, a JSON query language [21] to filter and extract JSON fields. This gives the user a set of tools to process JSON files. This is done by the function

$$\text{jmespath}(\text{expression}, \text{flag}). \quad (2)$$

The flags can be set as

1. *Natural* = 0 interpreted the natural expression,
2. *TypeName* = 1 selects the type name,
3. *Value* = 2 selects the value,
4. *ShortId* = 4 selects the short id,
5. *IdPath* = 8 selects the object of the id path.

For example, the following code snippet is explained. To give an essential example only the *Natural* flag is used. If the root is `{ "a": {b: "foo" } }` then the code

```

1      v = $
2      => jmespath(a,0)
3      v_2 = $v
4      => jmespath(b,0)
5      => exist()
```

assigns the value `{b: "foo" }` to `v`. Then it is checked if the extracted value `foo` of the JSON field `b` of variable `v` exists and assigns the boolean value `true` to `v_2`.

The AAS has a fixed formatted structure of subelements of a submodel. A simplified JSON example

```

1  [
2    {
3      "value":
4      [
5        {
6          "value": [
7            {
8              "shortId": "baz",
9              "value": "Hello",
10             "modelType": { "name": "string" }
11            }
12          ],
13          "shortId": "bar",
14          "modelType":
15          {
16            "name": "subElementCollection"
17          }
18        }
19      ],
20      "shortId": "foo",
21      "modelType":
22      {
23        "name": "subElementCollection"
24      }
25    }
26  ]
```

is given to show the need for AAS specialized functionalities of the DSL. Using the operator (2) with the flag *IdPath*, a functionality is implemented to extract the last object of a list of short ids separated by `"."`. The listing is translated to a natural expression to filter the submodel element collections structure of the submodel iterative by short id names. Using *foo.bar.baz* as an expression, it is

interpreted as follows

$$\begin{aligned} exp = & \{ \\ & r:[?shortId=='foo'] \\ & .value[?shortId=='bar'] \\ & .value[?shortId=='baz'] \\ & \}.r[0] \end{aligned}$$

a natural JMESPath expression. It filters an array of elements with the short id *foo* and gets the value fields. This is repeated with the short id *bar*. Then the last element with the short id *baz* of the JSON array is extracted. In the example above, the expression for this object is evaluated:

1	{
2	"shortId ":" baz ",
3	"value ":" Hello ",
4	"modelType ": { "name ":" string " }
5	}.

The examples show that the integration of JMESPath in the DSL creates a powerful tool to process JSON AAS structures. The DSL is very flexible and can be easily enhanced with more features. Therefore, there are functionalities to check a value of existent, get the value or value type of a submodel etc. Planned but not yet implemented is a regular expression functionality for mapping JSON fields and some auto type conversion patterns.

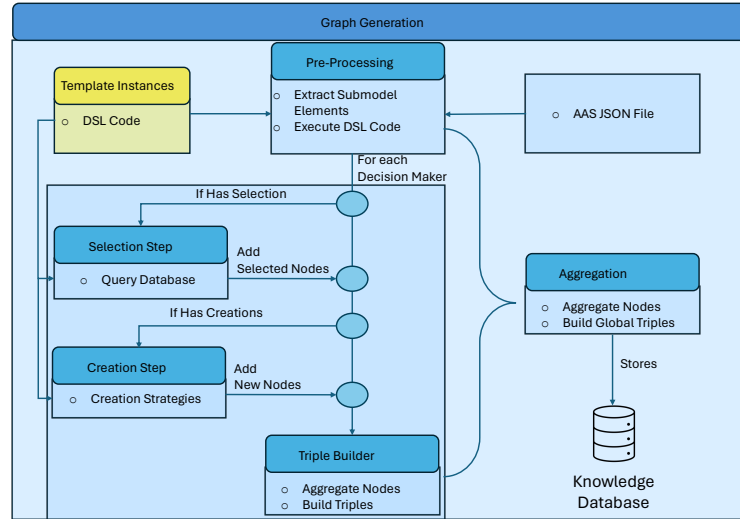


Fig. 6. Graph Generation Algorithm Structure.

4.4 Automatic Graph Generation Algorithm

In this section, the Graph Generation Algorithm is described. The different strategies to process AAS submodels are configured in the template instances of the Database. The DSL code preprocesses the AAS file and native queries are stored there. Furthermore, multiple decision-making processes are configured. They are made of selection and creation strategies using the native queries and the DSL. An overview is given in Figure 6.

As a precondition, the parameter of the global URL of the asset is set as an identifier of the asset shell to register. First in a pre-processing step the Submodel Elements are extracted from the AAS file. Then they are processed using the DSL to build a dictionary of variables as keys to aggregated properties. This dictionary is then used as a base data set for the decision-making processes.

Then for each configured decision maker a selection step to query existing entries of the database and a creation step to build new individual structures. Both aggregate the nodes of the new shell knowledge graph but only one of them must be configured. For each node it is necessary to have the aggregated information of the following described fields:

1. **url**: The global URL as identifier of the node.
2. **label**: The human readable label of the node.
3. **subclass url**: The identifier URL of the OWL entity, the node of which is a subclass.
4. **node name**: The node name as a short identification.
5. **properties**: A JSON dictionary with properties of the node to store.

Therefore, the selection step requires that the executed query returns a list of rows containing these fields.

In the creation step, strategies are configured to generate the values for each of the required fields, using the aggregated information from pre-processing and selection. Each strategy can be configured by the DSL but there are various native strategies. For example, there is a native strategy to attach a path to the asset or server URL to have a valid directory structure. This strategy transfers the asset shell structure to the knowledge database and guarantees an easy-to-query identifier.

The aggregated nodes are then processed in a triple builder to connect them with labeled edges. However, before a new edge is created it is checked if one of the connected nodes is a newly created node. If not, it is checked if such a connection already exists. In this step, all local duplicates are removed.

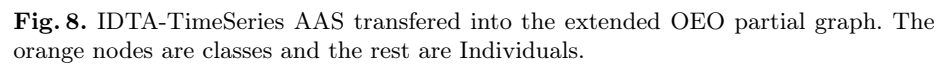
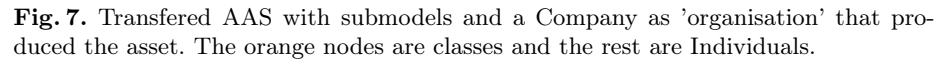
Then all nodes and edges are gathered and aggregated. In particular, the individual entries that are a subclass of categories asset, AAS and their associated submodels are generated. The removal of duplicate nodes and edges of graphs generated by different submodels, guarantees a coherent graph of the asset.

4.5 Generated Examples

For example, two shells are migrated into the knowledge base ontology. The submodel templates utilized are IDTA-Nameplating and IDTA-TimeSeries. The previously described functionalities store the structure of submodels data properties in their corresponding nodes. The corresponding graph of a nameplating is presented in Figure 7. It shows an asset that is represented by an AAS and its submodels including the submodel with IDTA-Nameplate structure. The producer of the asset is mapped as an organization in the knowledge graph.

The second example is a time series that fulfills the IDTA template. A part of the corresponding knowledge graph is shown in Figure 8, depicts the AAS and its corresponding typical metadata like resolution, start and end time and the measurement units.

Both examples show that the stored information of the AAS is semantically annotated and accessible to the user. Particularly in the area of time series, the data broker makes it possible to offer these to the user in an AAS industry standard format. This is of great value for the market for digital goods, as data sets are precisely specified and can thus facilitate the modeling and training of neural networks.



5 Conclusion

In summary, it is shown that DT descriptions of AAS and an ontology knowledge base can be combined to have the advantages of both. With the transfer of the knowledge base to a graph database a wide range of mathematical tools is enabled. So it can be shown that LLM text mining algorithms assist the user. The data broker application is also more scalable, especially concerning cloud technologies that use cooperative data spaces like Gaia-X [22]. Semantic interoperability makes it possible to search the offers in a targeted manner.

It is also shown that the mapping algorithm from 4.1, it works without losing class description information and the newly developed DSL presented in 4.3 is a tool to bridge the gap between the knowledge of a normal user and an ontology domain expert. It has a direct application in the configuration of the templates as a configuration tool for the automatized process of registering DTs, see Figure 9.

6 Discussion

The registering process based only on JSON files is no restriction because the tools of generation AAS exchange format files, like the AASX package explorer [23], have the functionality to convert the shell of an asset into a JSON representation. However, integrating the newly developed client technology of the AAS environment of the BaSyx project [24] can help to enable new promising features having direct access to the digital twins.

Another aspect is the integration of semantic ids shown in [9] and [12]. It can enhance the mapping process of an AAS file to a specific template. However, since there is no effective restriction on semantic IDs it requires a lot of specific domain knowledge. This will be the subject of future works.

In the assisted register process it is shown that AAS can be analysed by LLMs. However, due to the lack of a large data corpus, classical approaches cannot be dispensed with and generation without the help of the user is not possible. Nevertheless, the combination of LLMs, text mining, semantic descriptions and reasoning methods gives developers a lot of possibilities. As future work, it is interesting to examine if it is possible to transfer a native description to a query like it is done in a tourism application [25].

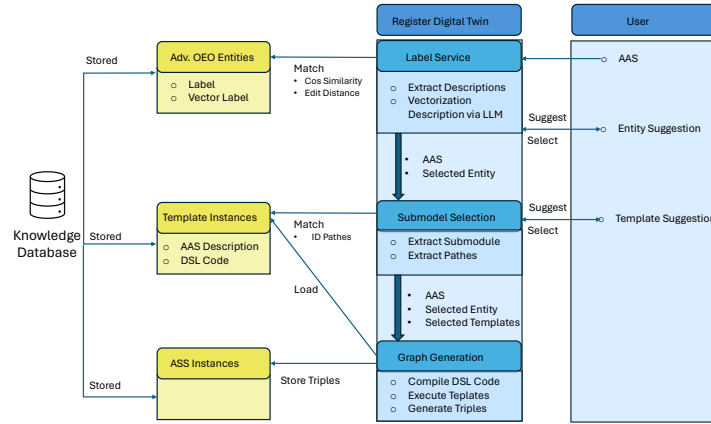


Fig. 9. Overview over the assisted digital twin register process.

Acknowledgments. This work has been funded by the Federal Ministry for Economic Affairs and Climate Action in the context of the Project IdFlexNetz (FKZ 03EI6067A-E). Special thanks to Julia Bayer and Daniel Rohrbach for corrections and to our project partners VSE AG, Schneider Electric and Spheryty.

References

- Grüner S., Neidig J., Orzelski A., Pollmeier S.: Asset Administration Shell Reading Guide, https://industrialdigitaltwin.org/wp-content/uploads/2022/11/2022-11-03_IDTA_AAS-Reading-Guide.pdf, March 2023, last accessed 2024/04/12.
- Booshehri M., Emele L., Flügel S., Förster H., Frey J., Frey U., Glauer M., Hastings J., Hofmann C., Hoyer-Klick C., Hülk L., Kleinau A., Knosala K., Kotzur L., Kuckertz P., Mossakowski T., Muschner C., Neuhaus F., Pehl M., Robinius M., Sehn V., Stappel M.: Introducing the Open Energy Ontology: Enhancing data interpretation and interfacing in energy systems analysis. In: Energy and AI, Volume 5, 100074, ISSN 2666-5468, 2021, <https://doi.org/https://doi.org/10.1016/j.egyai.2021.100074>.
- Meiser M., Duppe B., Zinnikus I.: Generation of meaningful synthetic sensor data — Evaluated with a reliable transferability methodology. In: Energy and AI, Volume 15, 2023, <https://doi.org/https://doi.org/10.1016/j.egyai.2023.100308>.
- Brandherm B., Deru M., Ndiaye A., Kiefer G., Baus J., Gampfer R.: Integration of Renewable Energies—AI-Based Prediction Methods for Electricity Generation from Photovoltaic Systems. In: Thomas Barton, Christian Müller, Apply Data Science: Introduction, Applications and Projects, Pages 137-158, ISBN 978-3-658-38798-3, Springer Fachmedien, Wiesbaden, Germany, GER, 2023, https://doi.org/https://doi.org/10.1007/978-3-658-38798-3_9.
- Schmeyer T. A., Kiefer G., Brandherm B., Klimenko A., Krämer K., Deru M., Ndiaye A., Baus J., Winter A., Igel M.: Assistance System for AI-based Monitoring and Prediction in Smart Grids. In: Human Computer Interaction International 2023, Human Computer Interaction International Conferences (HCII-2023), Springer Cham, 2023, https://link.springer.com/chapter/10.1007/978-3-031-36001-5_65.

6. European Commission: Accelerating the Digital Transformation of the European Energy System, <https://digital-strategy.ec.europa.eu/en/policies/digitalisation-energy>, last accessed 2024/04/12.
7. Ismail F. B., Al-Faiz H., Hasini H., Al-Bazi A., A. Kazem H.: A comprehensive review of the dynamic applications of the digital twin technology across diverse energy sectors. In: Energy Strategy Reviews, Volume 52, 101334, ISSN 2211-467X, 2024, <https://doi.org/https://doi.org/10.1016/j.esr.2024.101334>.
8. Bader S., Maleshkova M.: The Semantic Asset Administration Shell, 2019, https://doi.org/10.1007/978-3-030-33220-4_12.
9. Huang Y., Dhouib S., Medinacelli L. P., Malenfant J.: Enabling semantic interoperability of asset administration shells through an ontology-based modeling method. In: Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings (MODELS '22) Association for Computing Machinery, New York, NY, USA, Pages 497–502, 2022, <https://doi.org/https://dl.acm.org/doi/10.1145/3550356.3561606>.
10. Rongen S., Nikolova N., van der Pas M.: Modelling with AAS and RDF in Industry 4.0. In: Computers in Industry, Volume 148, 103910, ISSN 0166-3615, 2023, <https://doi.org/https://doi.org/10.1016/j.compind.2023.103910>.
11. Lehnert C., Engel G., Steininger H., Drath R., Greiner T.: A Hierarchical Domain-Specific Language for Cyber-physical Production Systems Integrating Asset Administration Shells. In: IEEE 26th International Conference on Emerging Technologies and Factory Automation (ETFA), Vasteras, Sweden, SWE, Pages 01-04, 2021.
12. Xia Y., Xiao Z., Jazdi N., Weyrich M.: Generation of Asset Administration Shell with Large Language Model Agents: Towards Semantic Interoperability in Digital Twins in the Context of Industry 4.0, March 2024, <https://arxiv.org/abs/2403.17209>.
13. Luxenburger A. , Porta D., Knoch S., Mohr J., Schwartz T.: A Service Infrastructure for Industrie 4.0 Testbeds based on Asset Administration Shells. In: IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), Sinaia, Romania, ROU, Pages 1-8, 2023, <https://doi.org/10.1109/ETFA54631.2023.10275335>.
14. Industrial Digital Twin Association e.V., AAS-Specs: Resource Description Framework, <https://github.com/admin-shell-io/aas-specs/tree/master/schemas/rdf>, 2024/05/29.
15. Spear A., Ceusters W., Smith B.: Functions in Basic Formal Ontology. In: Applied Ontology, Pages 103-128, 2016.
16. Industrial Digital Twin Association: IDTA Submodel Templates published, <https://industrialdigitaltwin.org/en/news-dates/publication-of-idta-submodel-templates-4071>, last accessed 2024/06/03.
17. Neo4j: neosemantics (n10s): Neo4j RDF & Semantics toolkit, <https://neo4j.com/labs/neosemantics/> last accessed 2024/05/16.
18. Singhal A. and I. Google: Modern Information Retrieval: A Brief Overview. In: IEEE Data Engineering Bulletin. Pages 35-44 Volume 24. 2021
19. Hugging Face: bert-base-uncased, <https://huggingface.co/google-bert/bert-base-uncased>, last accessed 2024/05/17.
20. Bechhofer S., van Harmelen F., Hendler J., Horrocks I., McGuinness D. L., Patel-Schneider P. F., Andrea Stein L., Olin F. W.: W3C-OWL Web Ontology Language Reference, <https://www.w3.org/TR/owl-ref/>, last accessed 2024/05/17.
21. James Saryerwinnie: JMESPath: JMESPath Libraries, <https://jmespath.org/libraries.html>, last accessed 2024/05/23.

22. National Academy of Science and Engineering: Gaia-X Hub Germany, <https://gaia-x-hub.de/en/>, last accessed 2024/05/23.
23. Eclipse: GitHub: AASX Package Explorer, <https://github.com/eclipse-aaspe/package-explorer>, last accessed 2024/05/24.
24. Eclipse BaSyx™: GitHub: eclipse-basyx, <https://github.com/eclipse-basyx>, last accessed 2024/05/24.
25. Chase A., Chikobava M., Deru M., Hauck C., Poller P., Ndiaye A., Schmeyer T. A., Gampfer R., Brandherm B.: Multimodales Fenster in die Vergangenheit der ehemaligen Vau-ban-Festung Saarlouis mittels ChatGPT. In: Barton T., Müller C.: Angewandte Wirtschaftsinformatik - Generative KI im Kontext der Wirtschaftsinformatik, Pages 1-19, Springer Vieweg, Wiesbaden, Germany, GER, December 2024. [to appear]